--- SECTION 1: NODE.JS CORE ---

1. JavaScript Basics

```
// 1. Hello World
console.log("Hello World");

// 2. Factorial of a Number
function factorial(n) {
  let result = 1;
  for(let i = 2; i <= n; i++) {
    result = result * i;
  }
  return result;
}
console.log(factorial(5));

// 3. Reverse a String
function reverseString(str) {
  let reversed = str.split("").reverse().join("");
  return reversed;
}

// 4. Prime Number Check
function isPrime(n) {
  if (n <= 1) return false;
  for (let i = 2; i < n; i++) {
    if (n % i === 0) return false;
  }
  return true;
}

// 5. GCD / HCF of two numbers
function findGCD(a, b) {
  while (b !== 0) {
    let temp = b;
    b = a % b;
    a = temp;
  }
  return a;
}
```

2. Functions & Modules

```
// --- myModule.js ---
exports.addNumbers = function(a, b) {
  return a + b;
};

// --- app.js ---
const myModule = require('./myModule.js');

// Normal Function
function showNormal(text) {
  console.log("Normal: " + text);
}

// Arrow Function
const showArrow = (text) => {
  console.log("Arrow: " + text);
};

let sum = myModule.addNumbers(10, 20);
console.log("Sum is: " + sum);
```

3. File Operations (fs)

```
const fs = require('fs');

// Write to a file synchronously
fs.writeFileSync('data.txt', 'Hello Full Stack!');
console.log("File Created & Written");

// Read from a file synchronously
let fileData = fs.readFileSync('data.txt', 'utf8');
console.log("File Contents: " + fileData);
```

4. HTTP Server & Query String

```
const http = require('http');
const url = require('url');

const server = http.createServer(function(req, res) {
  // Parse the request URL
  let parsedUrl = url.parse(req.url, true);

  res.writeHead(200, {'Content-Type': 'text/html'});

  // Static/Dynamic Routing Logic
  if (parsedUrl.pathname === '/login') {
    res.end('<h2>Login Page</h2>');
  }
  else if (parsedUrl.pathname === '/query') {
    // Reading data from query string: ?name=User
    let userName = parsedUrl.query.name;
    res.end('Hello ' + userName);
  }
  else {
    res.end('Welcome to Basic Web Server');
  }
});

server.listen(3000, function() {
  console.log("Server listening on port 3000");
});
```

5. Events & Callbacks

```
const events = require('events');
const eventEmitter = new events.EventEmitter();

// Create an event handler (Callback)
function myEventHandler(userName) {
  console.log("Event triggered for: " + userName);
}

// Assign the event handler to an event
eventEmitter.on('userLogin', myEventHandler);

// Fire the 'userLogin' event
eventEmitter.emit('userLogin', 'Chethan');
```

6. Node.js MongoDB (CRUD & Aggregation)

```
// VS Code Executable Node.js Script for MongoDB
const { MongoClient } = require('mongodb');

async function runMongoOps() {
  const uri = "mongodb://localhost:27017";
  const client = new MongoClient(uri);

  try {
    await client.connect();
    const db = client.db("fsd_lab");
    const coll = db.collection("students");

    // 1. Create (Insert)
    await coll.insertOne({ name: "Chethan", marks: 90 });
    console.log("Document Inserted");

    // 2. Read (Find)
    const student = await coll.findOne({ name:
"Chethan" });

    // 3. Update
    await coll.updateOne(
      { name: "Chethan" },
      { $set: { marks: 95 } }
    );

    // 4. Count, Sort, Skip, Limit Pipeline
    const total = await coll.countDocuments();
    const results = await coll.find()
      .sort({ marks: -1 })
      .skip(0)
      .limit(5)
      .toArray();

    console.log("Total Count:", total);
    console.log("Sorted Results:", results);

    // 5. Delete
    await coll.deleteOne({ name: "Chethan" });

  } finally {
    await client.close();
  }
}

runMongoOps().catch(console.dir);
```

--- SECTION 2: EXPRESS.JS ---

7. Express Routing & Params

```
const express = require('express');
const app = express();

// Basic Default Route
app.get('/', function(req, res) {
  res.send('Hello World from Express App!');
});

// Route with URL Parameters
app.get('/user/:id', function(req, res) {
  let requestedId = req.params.id;
  res.send('You requested data for User ID: ' +
requestedId);
});

// POST Route simulation
app.post('/submit', function(req, res) {
  res.send('Form Submitted Successfully');
});

// Start Express Server
app.listen(3000, function() {
  console.log("Express Server running on port 3000");
});
```

--- SECTION 3: ANGULAR & REACT ---

8. AngularJS (Directives & Bindings)

```
<!DOCTYPE html>
<html>
<head>
  <script src="angular.js"></script>
</head>
<body>

<!-- Initialize App and Controller -->
<div ng-app="myAngularApp" ng-controller="myCtrl">

  <!-- Two-Way Data Binding using ng-model -->
  <label>Enter Name:</label>
  <input type="text" ng-model="studentName">

  <!-- One-Way Data Binding using expressions -->
  <h1>Welcome to FSD Lab, {{ studentName }}!</h1>

</div>

<script>
  // Define Angular Module
  let app = angular.module("myAngularApp", []);

  // Define Controller & Scope
  app.controller("myCtrl", function($scope) {
    // Initial State
    $scope.studentName = "Mutlakunta Chethan";
  });
</script>

</body>
</html>
```

9. ReactJS (Components & Forms)

```
import React, { Component, useState } from 'react';

// 1. Functional Component using Props
function GreetingComponent(props) {
  return <h1>Hello, {props.name}</h1>;
}

// 2. Class Component with Lifecycle Methods
class DemoClassComponent extends Component {
  componentDidMount() {
    // Triggers automatically when component loads
    console.log("Component has been mounted to DOM!");
  }

  render() {
    return <h2>This is a Class Component</h2>;
  }
}

// 3. Form Validation Component with State Hooks
function UserForm() {
  // Define state for input
  const [inputValue, setInputValue] = useState("");

  // Handle Form Submission
  function handleFormSubmit(event) {
    event.preventDefault(); // Prevents page reload

    // Validation Logic
    if (inputValue === "") {
      alert("Error: Input field cannot be empty.");
    } else {
      alert("Success! Form Submitted: " + inputValue);
    }
  }

  return (
    <form onSubmit={handleFormSubmit}>
      <label>Enter Data: </label>
      <input
        type="text"
        value={inputValue}
        onChange={function(e)
{ setInputValue(e.target.value); }}
      />
      <button type="submit">Submit Form</button>
    </form>
  );
}
```